Approximate Multiply–Accumulate Units for Energy-Efficient DNN Accelerators

Error Characterization and Application-Level Evaluation

Sina Mozaffari Rad
Fatemeh Dastyafteh

Co-developed project | https://github.com/GitCodingMates/approx-mac-dnn

November 2026

Abstract

Multiply–accumulate (MAC) operations dominate both the computation and the energy budget of deep neural network (DNN) inference. Because DNNs are inherently tolerant to small numerical errors, the exact multiplier inside each MAC can be replaced with a cheaper *approximate* multiplier. This project studies that trade-off for 8-bit signed arithmetic. Bit-accurate models of an exact array multiplier, three column-truncated multipliers, and Mitchell's logarithmic multiplier were developed and characterized exhaustively over all 2^{16} input pairs. A fully-connected network was trained on MNIST, quantized to int8 with int32 accumulation, and evaluated with every multiplication routed through each approximate design. Results show that even a multiplier with a mean relative error of 23.9 % reduces classification accuracy by only 0.42 %, confirming that large hardware savings are available at negligible application-level cost.

1 Introduction

The processing elements (PEs) of a DNN accelerator are, at their core, MAC units. In an 8-bit datapath the multiplier is the largest and most power-hungry part of the PE, so its cost scales directly with the size of the PE array. *Approximate computing* exploits the error resilience of DNNs: errors in individual products are partially averaged out across long dot products, and the final classification decision (an `argmax`) is insensitive to small perturbations of the output scores.

The goals of this project are:

1. to build bit-accurate, hardware-oriented models of representative approximate multipliers;
2. to characterize their arithmetic error with standard metrics;
3. to measure their impact on the accuracy of a quantized neural network;
4. to implement them in synthesizable Verilog and verify the RTL exhaustively against the reference models;
5. to quantify the hardware savings (in progress, Section 7).

2 Multiplier Designs

All designs operate on 8-bit two's-complement operands $a, b \in [-128, 127]$ and are implemented in sign-magnitude form: the magnitudes $|a|, |b| \in [0, 128]$ are multiplied and the sign is restored as $\mathrm{sgn}(a) \oplus \mathrm{sgn}(b)$. This makes the error of every approximate design symmetric around zero.

2.1 Exact array multiplier (baseline)

The product is the sum of N^2 partial-product bits,

$$P = \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} a_i \, b_j \, 2^{i+j}, \qquad N = 8. \tag{1}$$

2.2 Column-truncated multiplier

All partial-product bits whose weight falls below column k are removed:

$$P_{\text{trunc}}(k) = \sum_{i+j \,\geq\, k} a_i \, b_j \, 2^{i+j}. \tag{2}$$

Each removed bit eliminates one AND gate and its share of the carry-save adder tree. For $k = 4, 6, 8$, respectively 10, 21 and 36 of the 64 partial-product bits are removed. No error-compensation constant is added, so the design always under-estimates the product magnitude.

2.3 Mitchell logarithmic multiplier

Writing each operand as $x = 2^k(1 + f)$, where $k = \lfloor \log_2 x \rfloor$ is found by a leading-one detector (LOD) and $0 \leq f < 1$, Mitchell's algorithm [1] approximates $\log_2(ab) \approx k_1 + k_2 + f_1 + f_2$. With $x_i = |a_i| - 2^{k_i}$, an exact-integer form suitable for hardware is

$$S = x_1 2^{k_2} + x_2 2^{k_1}, \qquad P_{\text{Mitchell}} = \begin{cases} 2^{k_1+k_2} + S, & S < 2^{k_1+k_2} \\ 2S, & \text{otherwise.} \end{cases} \tag{3}$$

The datapath therefore consists only of two LODs, barrel shifters and a single adder; no partial-product array is required. The design always under-estimates, with a worst-case relative error of about 11.1 %.

3 Methodology

3.1 Bit-accurate reference models

Each design is described in Python/NumPy at the bit level (`python/approx_mult.py`), mirroring the intended RTL structure (partial-product selection, LOD, shifts). These models serve as the golden reference for RTL verification. For fast evaluation, each model is expanded into a 256×256 product look-up table.

3.2 Error metrics

Every design is evaluated over all 65,536 input pairs. With P the exact and $\hat{P}$ the approximate product, and $\text{ED} = |\hat{P} - P|$, the following metrics are reported [2]:

- **ER** – error rate, fraction of inputs with ED $\neq 0$;
- **MED** – mean error distance, $\mathbb{E}[\text{ED}]$;
- **NMED** – MED normalized by the maximum product (128^2);
- **MRED** – mean relative error distance, $\mathbb{E}[\text{ED}/|P|]$ over $P \neq 0$;
- **Max ED** – worst-case absolute error.

3.3 Quantized neural network

A 784–128–10 multilayer perceptron with ReLU activation was trained on MNIST [5] for eight epochs using mini-batch SGD (batch 128, learning rate 0.1), implemented directly in NumPy. The trained network was then quantized following the integer-only inference scheme of [4]:

- inputs: $x_q = \mathrm{round}(127\,x)$, $x \in [0, 1]$;
- weights: symmetric per-tensor int8, $s_w = \max|W|/127$;
- biases: int32 at scale $s_x s_w$;
- accumulation: int32;
- hidden activations: requantized to unsigned 7-bit values using a scale calibrated on 5,000 training images (99.99th percentile).

During approximate inference, *every* multiplication in both layers is replaced by the corresponding approximate multiplier, exactly as it would occur inside the accelerator's PEs. Accuracy is measured on the full 10,000-image test set.

4 Results

4.1 Arithmetic error

Table 1 summarizes the exhaustive error characterization, and Figure 1 shows how the relative error is distributed over the input space.

Table 1: Error characterization over all 2^{16} signed 8-bit input pairs.

Multiplier	ER (%)	MED	NMED (%)	MRED (%)	Max ED
Exact	0.00	0.00	0.000	0.00	0
Truncated k=4	81.25	12.25	0.075	1.72	49
Truncated k=6	93.75	80.25	0.490	7.59	321
Truncated k=8	98.05	384.75	2.348	23.87	1537
Mitchell	87.89	151.34	0.924	3.70	1024

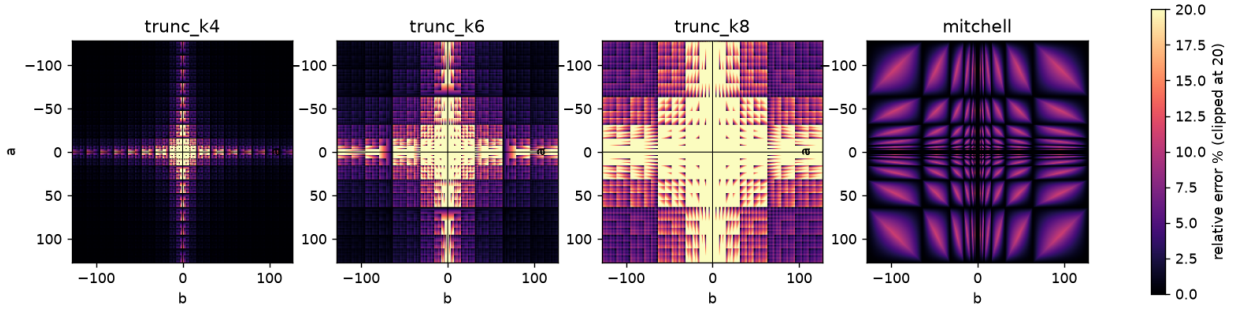


Figure 1: Relative error (%) of each approximate multiplier as a function of its operands. Truncation errors concentrate at small operands, where the removed low-order columns carry a larger share of the product; Mitchell's error depends on the fractional parts of both operands and is bounded at about 11 %.

The measured MRED of the Mitchell multiplier (3.70 %) agrees with the value reported in the literature (≈3.8 %), which validates the model. Truncation exhibits a clear knob: each increase of k by two columns raises the MED by roughly a factor of five.

4.2 Application-level accuracy

Table 2 shows the central result of this project: arithmetic error and application error are only weakly coupled. Int8 quantization itself costs just 0.01 %. Truncation up to $k = 6$ and the Mitchell multiplier are practically lossless, and even $k = 8$ truncation—which removes more than half of the partial-product array and has an MRED above 20 %—loses less than half a percentage point. Two effects explain this resilience:

Table 2: MNIST test accuracy of the int8 MLP with each multiplier. Float32 reference: 96.82 %.

Multiplier	Accuracy (%)	Δ vs. exact int8
Exact (int8)	96.81	–
Truncated k=4	96.82	+0.01
Truncated k=6	96.77	−0.04
Truncated k=8	96.39	−0.42
Mitchell	96.77	−0.04

1. **Error averaging.** Each output neuron accumulates up to 784 products; independent errors partially cancel, so the relative error of the sum is far smaller than that of a single product.
2. **Decision robustness.** Classification depends only on the ranking of the output scores, so a systematic, roughly proportional under-estimation (as produced by both truncation and Mitchell) rarely changes the `argmax`.

5 RTL Implementation

All designs are written in synthesizable Verilog-2001 (`rtl/`) and share a common `sign_mag` front end that converts the two's-complement operands to 8-bit magnitudes and computes the product sign.

- `mul_exact` – baseline 8×8 magnitude multiplier.
- `mul_trunc #(K)` – the partial-product array is built row by row; in row j the bits of $|a|$ below position $K - j$ are masked with an elaboration-time constant, so the corresponding AND gates and adder inputs are removed by synthesis rather than gated at run time. A single parameter covers the whole design space.
- `mul_mitchell` – two priority-encoder LODs, mantissa extraction by clearing the leading one, two barrel shifters, one 16-bit adder and a comparator/multiplexer that selects between the two cases of Eq. (3).
- `mac #(MUL,K)` – a two-stage pipelined MAC. Stage 1 registers the product, stage 2 sign-extends it and accumulates into a 32-bit register. A `clr` input restarts the accumulation so consecutive dot products can be streamed without bubbles, and an `en` input supports idle cycles. The multiplier is selected at elaboration time with a `generate` block, so all variants share one PE description.

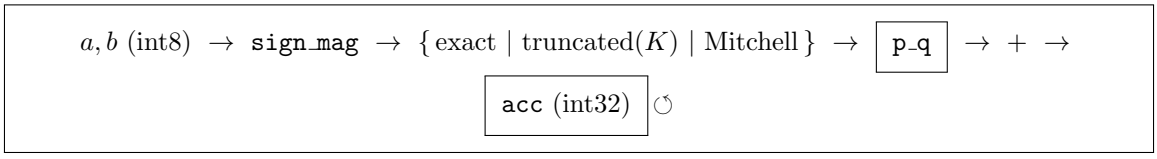


Figure 2: MAC datapath. Boxed signals are pipeline registers; the multiplier block is chosen by the `MUL` parameter.

6 Functional Verification

Verification reuses the Python models as the golden reference. `python/gen_vectors.py` exports, for each design, the expected product of all 2^{16} operand pairs as a hex table indexed by $\{a, b\}$. Two self-checking testbenches (`tb/`) were run in ModelSim; both are also compatible with Icarus Verilog.

- `tb_mul` applies all 65,536 operand pairs to the five multiplier instances simultaneously and compares every output bit-exactly with the golden table.

- `tb_mac` streams 20,000 random operand pairs into three MACs (exact, truncated K=6, Mitchell) with randomly inserted idle cycles and a `clr` every 784 terms, mimicking the first layer of the MLP. A cycle-accurate reference accumulator with the same two-cycle latency checks the output on every cycle.

Table 3: RTL verification results.

Testbench	Design	Checks	Mismatches
`tb_mul`	exact	65,536	0
	truncated K=4	65,536	0
	truncated K=6	65,536	0
	truncated K=8	65,536	0
	Mitchell	65,536	0
`tb_mac`	exact, trunc. K=6, Mitchell	20,005 cycles	0

The exhaustive comparison proved its value during development: the first Mitchell implementation failed on 31,193 of the 65,536 pairs. The cause was the shift amount $k_1 + k_2$, which Verilog evaluated with the 3-bit width of its operands and therefore wrapped for $k_1 + k_2 \geq 8$. Widening the sum to 4 bits fixed the design. Such width-dependent errors appear only for specific operand ranges and are easily missed by random testing, but cannot escape an exhaustive check.

7 Work in Progress

1. **Hardware cost.** Logic synthesis with Yosys to compare cell count and estimated area, giving the final *area saving vs. accuracy loss* Pareto front.
2. **Extensions.** Error-compensated truncation, and evaluation on a convolutional network with a harder dataset.

8 Repository Structure

```
python/approx_mult.py bit-accurate multiplier models
python/error_analysis.py exhaustive error metrics and heatmaps
python/train_quantize.py NumPy MLP training and int8 quantization
python/approx_inference.py inference through approximate multipliers
python/gen_vectors.py golden vectors for RTL verification
rtl/ Verilog: sign_mag, mul_exact, mul_trunc,
                        mul_mitchell, mac
tb/ self-checking testbenches, run_sim.sh
results/ metrics (CSV), figures, quantized model
docs/report.tex this report
```

All experiments are reproducible with `pip install -r requirements.txt`, running the Python scripts in the order listed above, and `sh tb/run_sim.sh` for RTL simulation.

References

[1] J. N. Mitchell, "Computer multiplication and division using binary logarithms," *IRE Transactions on Electronic Computers*, vol. EC-11, no. 4, pp. 512–517, 1962.

[2] H. Jiang, F. J. H. Santiago, H. Mo, L. Liu, and J. Han, "Approximate arithmetic circuits: A survey, characterization, and recent applications," *Proceedings of the IEEE*, vol. 108, no. 12, pp. 2108–2135, 2020.

[3] V. Mrazek, R. Hrbacek, Z. Vasicek, and L. Sekanina, "EvoApprox8b: Library of approximate adders and multipliers for circuit design and benchmarking of approximation methods," in *Proc. DATE*, 2017, pp. 258–261.

[4] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. CVPR*, 2018, pp. 2704–2713.

[5] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.